

RetryGuard: Preventing Self-Inflicted and Attack-Driven Retry Storms in Cloud Applications

Jhonatan Tavori* Anat Bremler-Barr[§] Hanoach Levy[§] Ofek Lavi[§]
*Columbia University [§]Tel Aviv University

Abstract—Modern cloud applications are built from independent microservices, offering scalability and usage-based billing. However, their reliance on independently-operating auto-scalers introduces coordination challenges. Default retry patterns can trigger “retry storms” during service miscoordination or adversarial overload, amplifying load, latency, and resource billing. These storms can cause either self-inflicted Denial-of-Wallet (DoW) or amplify the impact of DDoS attacks.

To overcome these problems, we introduce *RetryGuard*, a distributed framework for productive control of retry patterns across interdependent microservices. By managing retry policy on a per-service basis and making parallel decisions, *RetryGuard* prevents retry storms, curbs resource contention, and mitigates escalating operational costs. *RetryGuard* makes its decisions based on an analytical model that captures the relationships among retries, throughput (rejections), delays, and costs. Simulations show that it outperforms established mechanisms, including exponential backoff, jitter, and retry budgets.

Experimental results show that *RetryGuard* significantly reduces resource usage and costs compared to existing AWS policies, achieving more than 90% improvement in latency, and 98% reduction in storm size. We further demonstrate its effectiveness in a multi-layer Kubernetes deployment with the Istio service-mesh, where *RetryGuard* reduces the peak number of replicas by 3× and cumulative memory usage by 55%.

I. INTRODUCTION

Retries are among the oldest fault-tolerance techniques in networking, dating back to ARPANET [1]. In today’s cloud environments, application-level retries, typically layered atop HTTP transports such as REST and gRPC, are enabled by default in most platforms [2], [3]. Yet, as demonstrated in this work, common cloud retry policies can trigger costly self-inflicted retry storms in microservice-based applications.

In recent years, cloud applications have shifted from monoliths to microservices-based architectures, decomposing functionality into smaller independent services with clear interfaces [4], [5], to improve scalability, manageability, and deployment flexibility [6]–[8]. Thanks to *auto-scaling* mechanisms, microservices can be deployed across diverse hardware configurations and are billed based on usage (e.g., per operation or per runtime), aligning with the *serverless* approach.

At face value, it appears as if microservices applications, supported by auto-scaling capabilities, would free operators from concerns about manual resource allocation [9]. Nonetheless, this flexibility introduces significant coordination challenges, posed by the interactions between independently developed and deployed services whose operation is interdependent. Consequently, the services may experience periods

of *miscoordination*¹, lasting several minutes, during which they may be subject to blocking (rejections) and delays. During periods of miscoordination, retry operations, which are the focus of this work, may become counterproductive.

Retry Operation under Miscoordination. Retries between services in tandem have been designed to perform the important task of overcoming *instantaneous* service blocking (at the downstream service) due to statistical variations of the request stream or random service errors. Their effectiveness generally relies on the services being well coordinated, namely the downstream service operates under *stability conditions* with load $\rho < 1$, where ρ is the total arrival rate, including retries, divided by the downstream service rate. Our key observation, however, is that when the services enter a period of *prolonged miscoordination*, during which the downstream service experiences load $\rho > 1$, their behavior changes drastically, and retries become counterproductive, exacerbating inefficiencies and inflating operational costs.

Such miscoordination is common in modern microservices-based cloud applications and may arise from: (1) *Independent auto-scalers*, causing delays in matching capacities between service tiers; (2) *Misconfiguration* errors, leading to persistent miscoordination [10]; and (3) *DDoS attacks*, which impose sustained overload through high volumes of malicious traffic.

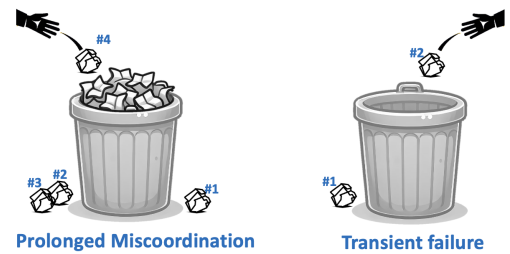


Figure 1: Prolonged versus instantaneous blocking.

Fig. 1 illustrates the target service as a trash basket. Under stable load, retries are likely to succeed; under prolonged miscoordination, unavailable capacity causes repeated failures, further increasing load, delay, and rejection.

¹We use *miscoordination* to denote a prolonged mismatch between the request rate of one tier and the serving capacity of another. This can arise from delayed scaling (e.g., an upstream service scales faster than its downstream dependency) or misconfigurations (e.g., manual scaling or concurrency limits).

Retry Storms: Self-Inflicted or Attack-Driven DoW. Excessive retries can escalate into costly and disruptive “retry storms”, a form of self-inflicted Denial-of-Service (DoS) scenario² [2], [11]. Figure 2 illustrates a retry storm. If Service A’s auto-scaler reacts faster than Service B’s, a surge in HTTP requests causes A to scale up while B lags behind, creating a *bottleneck*. As a result, blocked requests at B may accumulate at A, forming a *wave* awaiting completion. Service A, assuming the issue is temporary, retries the blocked requests multiple times, causing a snowball effect on the already struggling Service B. Each failed request consumes resources across multiple retry attempts before ultimately failing. Furthermore, when B eventually scales, it may overestimate demand based on the accumulated wave. Thus, the system experiences a self-inflicted Denial-of-Wallet (DoW) scenario: Retry-amplified requests load in the upstream services, as well as over-scaling at the downstream service. Both phenomena, which may last for a long duration, inflict significant costs on the application.

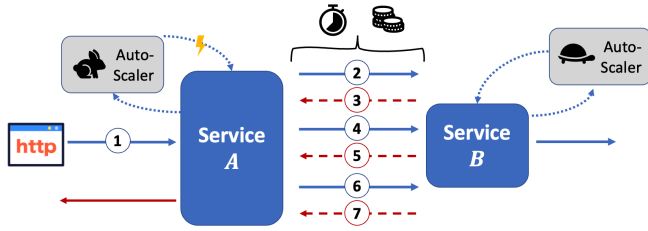


Figure 2: Self-Inflicted Retry Storm: Miscoordinated tandem services.

The same amplification can be triggered externally: A short DDoS burst can overload the downstream service, with retries sustaining the storm after the attack ends and turning it into a prolonged DoW condition.

RetryGuard: Optimizing Retries Control. Retry-amplified traffic highlights the need to adapt retry strategies to situations of miscoordination between services and to detect when retries become counterproductive. This adaptation is essential not only for maintaining stability and reducing costs, but also for improving overall performance. We present **RetryGuard**, a distributed mechanism designed to mitigate self-inflicted retry storms based on this principle. *RetryGuard* dynamically detects miscoordination, based on various parameters (including retry volume, rejections, or delays), and enables or disables retries at the service level accordingly. Our approach provides a generalizable framework applicable across diverse systems.

Unlike common retry strategies, including exponential backoff [12], jitter [13], retry budgets [14], and leaky buckets [15], *RetryGuard* enforces fundamentally distinct operational rules for transient failures and prolonged miscoordination. By detecting sustained mismatches between service tiers, it suppresses retries only when they become counterproductive,

²Self-inflicted DoS occurs when a service generates more traffic than it can handle, resulting in throttling, delays, or failures.

AWS	Retry Policy	Retries/Req	Rejection Rate	Resources Billing
	Legacy	2.09	19.8%	10.29×
	Standard	1.08	19.4%	3.29×
	Adaptive	0.52	19.6%	1.67×
	RetryGuard	0.05	18.8%	1.00×

Istio	Retry Policy	Retries/Req	Rejection Rate	Peak Replicas
	Default	0.31	5%	616
	RetryGuard	0.01	4%	209

Table I: Eliminating Superfluous Costs: Comparison of *RetryGuard* with AWS and Istio retry policies under 150–200% load during scaling. *RetryGuard* substantially reduces retry amplification, resource consumption, and associated costs while maintaining or improving the rejection rate.

offering a fundamentally different approach from legacy mechanisms. Their behavior and limitations are examined in Sections IV and VI. Compared with circuit breakers, *RetryGuard* targets the retry storm itself by suppressing only retries while allowing new requests to use the available capacity.

RetryGuard is supported by traffic analysis of a tandem system (Sec. V), which characterizes traffic properties and yields sharp differentiation between the situations of instantaneous failures and prolonged miscoordination. Experiments (Sec. VI) demonstrate its advantages over existing retry policies.

In Table I, we present a summary of *RetryGuard*’s performance in comparison to AWS and Istio retry policies, with a full evaluation detailed in Sec. VI. It summarizes *RetryGuard*’s performance³ and resource-cost improvements.

A. Main Contributions

We make the following key observations:

- Retries present challenges that are intrinsically linked to modern cloud applications.
- Under prolonged miscoordination and overload, retries turn out to be counterproductive. Existing retry solutions are ineffective in miscoordination scenarios.
- Economic considerations are critical in this context, while prior efforts have predominantly focused on performance.

The main contributions of this paper are as follows:

- Demonstrating self-inflicted DoW using an AWS Serverless application and an Istio-based Kubernetes application, showing its impact on latency, cost, CPU usage, and the number of provisioned replicas (Sec. III).
- Proposing and implementing **RetryGuard**, a distributed mechanism based on a productive-retry controller (see Sec. IV for design principles), and comparing it with exponential backoff, jitter, and retry budgets.
- Providing a mathematical model and cost analysis for the core of the problem (Sec. V), deriving the activation threshold for the productive-retry controller, and supporting its robustness and feasibility.

³Note that, naturally, some request failures are inevitable due to the capacity limits of bottleneck services (i.e., the pigeonhole principle) regardless of the retry strategy. However, *RetryGuard* minimizes additional failures.

- Deploying and experimentally evaluating *RetryGuard* against AWS’s retry APIs and Istio’s retry mechanisms in a Kubernetes service-mesh deployment (Sec. VI).
- Examining how retry mechanisms can be exploited by malicious users through DDoS attacks to trigger over-provisioning and prolonged request lifetimes, and discussing *RetryGuard* as a mitigation strategy (Sec. VII).

Additionally, we provide an overview of key cloud challenges, including coordination difficulties and the limitations of existing solutions (Sec. II). In Sec. VII we discuss the economic *interplay* between the cloud provider and the application owners (the cloud customers); concluding remarks are given in Sec. VIII.

II. BACKGROUND AND RELATED WORK

We discuss the limitations of existing mechanisms in handling retries under miscoordination and related prior work.

A. Background

Retry Mechanisms Are Integral to Cloud Applications. Retry mechanisms handle failures by transparently retrying failed requests for service [16], and have been used for decades as a resiliency mechanism in fault tolerant computer systems [1] and networks [14], [17], [18]. Microservices frequently interact over HTTP, HTTPS [19] or RPC [20] protocols, and the retries are embedded in the calling functions (provided by the SDKs). Many providers incorporate retry mechanisms into their libraries, offering automated retry implementations, and recommend traditional techniques like exponential backoff and jitter (e.g., AWS [13] and Microsoft Azure [17]). This approach is also common in service mesh platforms [11], [21]. A key feature of these platforms is their ability to automatically handle retries for failures [22], [23]. According to a recent survey by the CNCF, 56% of respondents identified reliability features, such as retries, as a key driver for adopting service mesh technology [24].

Collaborating Microservices Face Coordination Challenges. When multiple microservices collaborate to handle high-volume traffic, ensuring smooth coordination becomes a critical challenge, particularly if each service follows its own scaling policy. Auto-scaling, a key feature of cloud computing, automatically adjusts resources to accommodate changing workloads [25] and offers significant flexibility. However, since different services may scale in different ways [26], [27], this flexibility may also introduce miscoordination. For example, *databases* usually scale vertically (by provisioning larger instances) or through sharding, *virtual machines* rely on resizing or load balancing, and *Functions-as-a-Service* (FaaS) scale almost instantly. Miscoordination between the scaling of the services may lead to resource waste and increased budget consumption. Similar dependencies also arise in service meshes [28]. [29] highlighted budget waste caused by miscoordination, showing that customers often pay for doomed-to-fail requests.

However, they *did not* address retries.

Cost Tracking in the Cloud is Challenging. It is important to note that understanding cloud spending remains challenging due to complex billing and applications spanning hundreds or thousands of microservices [30]. Organizations frequently struggle to identify high-cost services, resulting in potential overspending⁴. In the scenarios highlighted in this paper, the lack of granular accurate tracking makes it much harder to monitor expenses and detect inefficiencies or costs caused by miscoordination and retry storms, often leaving operators unaware of spikes until after significant charges have accumulated.

B. Related Work

Optimization of cloud microservice architectures has been the focus of extensive recent work, with techniques such as circuit breakers, timeouts, and load controls. Prior work has addressed overload [32] through per-caller backoff [33], server-side admission control and token-based schemes [34]–[36], reinforcement learning [37], and credit-based admission [38]. Other works focused on server-side issues like CPU contention [35], [39]. These approaches primarily regulate offered load and do not specifically address retry storms caused by service miscoordination. *RetryGuard* instead removes the self-inflicted, retry-amplified portion of the load while preserving first attempts, leaving the underlying capacity shortfall to autoscaling or existing overload controllers. Thus, it is complementary to mechanisms such as [35], [38], [40].

Retry mechanisms are widely supported by cloud providers and service-mesh platforms, which offer guidance for avoiding common pitfalls, including Microsoft [2], Google Cloud [3], and Linkerd [11]. These recommendations typically control retry timing or rate. Exponential backoff [12] and jitter [13] delay and spread retries under the assumption that failures are transient, while Linkerd’s retry budgets [14] limit the retry rate. However, as discussed in Sec. IV, these mechanisms are ineffective under prolonged service-level miscoordination, where failures persist and the retries themselves become harmful. *RetryGuard* adds regime detection, treating sustained miscoordination as a distinct operating mode in which retries are suppressed while first attempts are preserved.

Despite the recent academic interest in microservices-based architectures, limited research exists on how retry patterns impact performance in cloud-based applications. Recently, [22] conducted experiments on a Kubernetes testbed using the Istio service mesh, demonstrating that retry configurations significantly influence application throughput and latency. [41], [42] showed similar findings using a probabilistic model checker. However, to the best of our knowledge, no systematic study has been conducted to study the behavior and economic impact of retries in the context of microservices miscoordination.

⁴Obtaining precise, time-specific cost data remains challenging as metrics tools often aggregate usage into coarse intervals [31] and some report only every few minutes, limiting visibility into sudden fluctuations.

III. CASE STUDIES: SELF-INFLICTED DOW

To demonstrate self-inflicted DoW resulting from retry storms, and its severity, we present two case studies: an AWS web application and a Kubernetes application with Istio service mesh. Both cases will demonstrate how, during miscoordination and overload conditions, retries can significantly **increase** resource billing costs for the application owner *while* potentially severely **degrading** user performance.

A. AWS Lambda and DynamoDB

We choose a popular AWS architecture, consisting of AWS Lambda, a serverless compute service (FaaS) [43], and AWS DynamoDB, a popular fully-managed database [44].

1) *Setup*: Each HTTP request triggers a single Lambda execution that performs a write operation to DynamoDB. The tested architecture is depicted in Fig. 3. Lambda and DynamoDB scale differently: Lambda scales nearly *instantaneously* by provisioning a separate instance for each request [43]. In contrast, DynamoDB (in provisioned mode) attempts to adapt to the measured arrival rate. This process scaling involves a *delay* due to the decision-making process and resource adjustment [44]. Throttled DynamoDB requests returned an error, prompting Lambda to decide on a retry. The Lambda function used AWS default retry configuration.

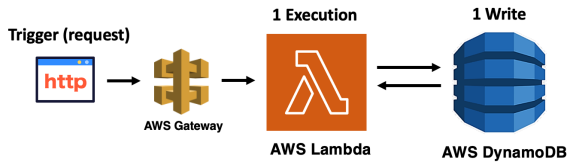


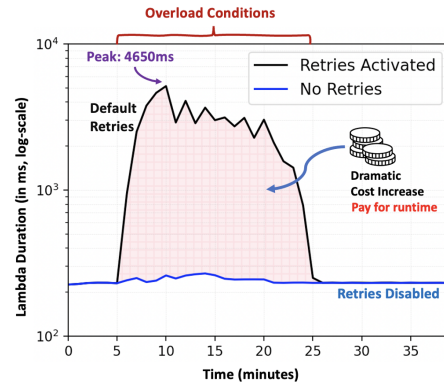
Figure 3: AWS Lambda and DynamoDB in tandem.

2) *Self-Inflicted DoW: Over-Scaling and Extensive Delays*: Measurements⁵ are depicted in Figs. 4a and 4b. Initially, the system was balanced. Then, at minute 5 the rate increased by 200%. While the Lambda function (blue curve, Fig. 4b) promptly adapted to it, DynamoDB did not⁶. DynamoDB took nearly 20 minutes to stabilize (red dotted curve, Fig. 4b) with multiple scale operations. During this period, a retry storm was triggered. Each failed DB operation forced the calling Lambda to wait and retry until success or timeout. During the DB scaling, Lambda's average latency (blue curve, Fig. 4a) spiked drastically to a 2,000% increase compared to the typical duration. Furthermore, due to the retry storm that overwhelmed it, the DB stabilized at 20% over-scaling (capacity of 448 req/sec instead of the required 383 operations/sec, 90% utilization, Fig. 4b, red dots).

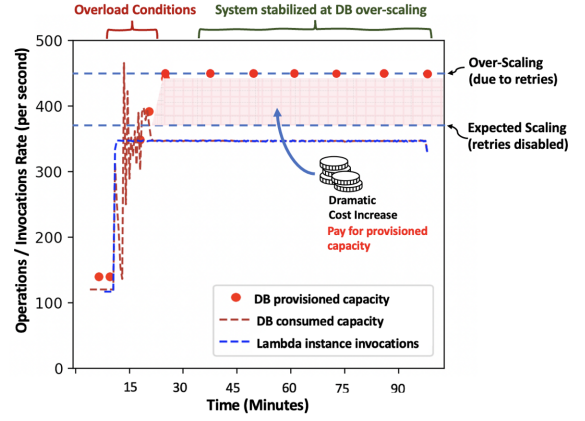
Repeating the experiment with retries turned off (blue curve, Fig. 4a) eliminated the excess delays, and no retry storm formed or over-scaling occurred.

⁵We collected data on Lambda invocations and DB operations and capacity using HTTP response codes and Amazon CloudWatch reports, a monitoring service by AWS that provides metrics, alarms, and more [31].

⁶As can be seen in the red dashed curve, there is an isolated jump right away due to the 'reserved capacity' feature, which is meant to handle sudden spikes, but after a few seconds it returned to its actual provisioned size.



(a) Lambda delay and cost (log scale): up to 2000% latency increase during retry storms.



(b) DB over-scaling: Over-provisioned (red dots) during and after the retry storm.

Figure 4: AWS: Retry storm and Over-provisioning.

3) *The Price Tag of Retry Storms*: Our experiments show that retry-based processes can substantially raise costs in two significant ways even after the storm: (1) Lambda's costs spiked by up to 2000% at peak load, with a substantial increase sustained throughout the 20-minute scaling period⁷. (2) DynamoDB costs rose by about 20% due to over-scaling⁸, with elevated charges potentially lasting for hours or days because of scale-down policies⁹.

B. Service Mesh: Istio's Bookinfo

Having shown the challenges of retries on AWS, we now demonstrate them with the Istio Service Mesh platform [45].

1) *Setup*: We used the Bookinfo application [21], a widely used microservices benchmark [23], composed of four services (see Fig. 5). The services used Kubernetes' default autoscaling [46]. The *Reviews* service had slower scaling decisions with a 60-second stabilization window. Istio's retry feature permitted

⁷Lambda costs are driven by invocations and latency.

⁸Provisioned-mode pricing depends on the provisioned capacity.

⁹AWS DynamoDB scales down only if utilization remains below a configured threshold for more than 15 consecutive minutes [44].

up to five attempts. We collected the experimental data using Prometheus [47].

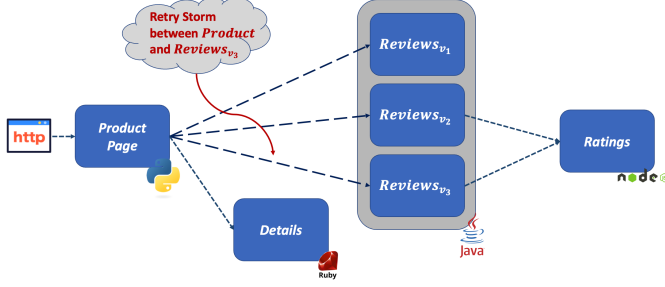


Figure 5: Istio Bookinfo: End-to-end architecture.

2) *Self-Inflicted DoW*: In this experiment (Fig. 6), the system initially processes traffic at a stable rate. At 300 sec, the rate surges. While the *Product* service scales up quickly, *Reviews*, with slower scaling, lags behind. This disparity triggers a “retry storm” (dashed red line, Fig. 6) as *Product* continuously retries *Reviews*. Due to the retry storm, *Reviews* eventually expands, inflicting significant over-charges¹⁰ (red continuous line, Fig. 6) for a period of 20 minutes.

We repeated the experiment with retries disabled. *Reviews* scaled to a level three times lower than its original scaling value (blue continuous line, Fig. 6) and completed its scale-up five minutes earlier. Furthermore, the overall rejection rate slightly decreased from 5% to 4%, and request latency was lower compared to the original setup by up to 30%.

Overall, as we observed in AWS, enabling retries during prolonged miscoordination conditions increased costs and degraded performance, offering no benefits on this test case.

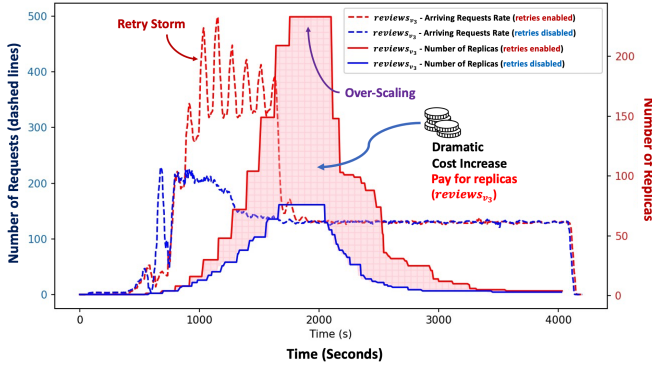


Figure 6: Bookinfo: Retry storm is formed at *Product* page.

IV. RETRYGUARD DESIGN

We next propose **RetryGuard**: a mechanism that dynamically enables or disables retries at the *service level*, based on a *productive-retry* controller which detects whether retries are productive (instantaneous failures) or counterproductive (miscoordination). The design is supported by the analysis in Sec. V.

¹⁰As with the AWS experiment, the system experiences drastic increases of request latencies; for the sake of brevity their analysis is omitted here.

A. A Dynamic Controller for Productive Retries

The productive-retry controller evaluates retry effectiveness at the service level using observable performance metrics: retry volume, request responses and error codes (rejections), and execution durations (delays). Alg. 1 presents the pseudo-code of a rejection-based variant, where *measure_value()* collects failure signals through monitoring systems such as AWS CloudWatch [31] or Google Cloud Monitoring [48], [49]. If the measured rejection rate remains above a predefined threshold for a sustained interval, retries are disabled; once it remains below the threshold, retries are re-enabled.¹¹

Algorithm 1 Rejection-based productive-retries controller

```

1: Set LowStreak  $\leftarrow$  0, HighStreak  $\leftarrow$  0, Retries  $\leftarrow$  ON
2: Set Threshold (parameter) and Interval (parameter)
3: while true do
4:   Failures  $\leftarrow$  measure_value()
5:   if Failures < Threshold then
6:     LowStreak  $\leftarrow$  LowStreak + 1
7:     HighStreak  $\leftarrow$  0
8:   else if Failures > Threshold then
9:     HighStreak  $\leftarrow$  HighStreak + 1
10:    LowStreak  $\leftarrow$  0
11:   else
12:     LowStreak  $\leftarrow$  0 and HighStreak  $\leftarrow$  0
13:   if LowStreak  $\geq$  Interval then Retries  $\leftarrow$  ON
14:   else if HighStreak  $\geq$  Interval then Retries  $\leftarrow$  OFF

```

Note that *RetryGuard*’s actions are active only during prolonged periods. This service-layer operation reflects a philosophy similar to BBR [50], which proposes to proactively adjust traffic at the transport layer. Thus, in well-scaled or instantaneous errors, *RetryGuard* matches the performance of legacy mechanisms, while during miscoordination, it outperforms them by preemptively preventing nonproductive attempts.

B. Comparison to Common Retry Mechanisms

Conventional retry strategies such as exponential backoff and jitter primarily control retry timing by delaying or randomizing subsequent attempts. While effective for transient failures, under sustained miscoordination they only spread redundant retries over time and may substantially increase request latency. As shown in Fig. 7(a) and (b), both mechanisms continue generating large retry volumes and high latency throughout the overload period.

Retry budgets appear more suitable because they explicitly limit retry volume. However, they still permit retries during prolonged overload, even when such attempts are unlikely to succeed. Figures 7(a) and (b) show that retry budgets reduce amplification relative to backoff and jitter, while panels (c) and (d) examine several budget values in greater detail. Reducing the budget (0.25 to 0.1) consistently lowers retry volume and latency, but does not eliminate redundant attempts. *RetryGuard* instead detects the sustained-overload regime and disables

¹¹Alg. 1 presents the binary ON/OFF variant; the controller extends naturally to gradual re-enabling, where retries resume at a fraction of their original rate and ramp up while rejections stay below the threshold, as used in our Istio deployment (Sec. VI-B).

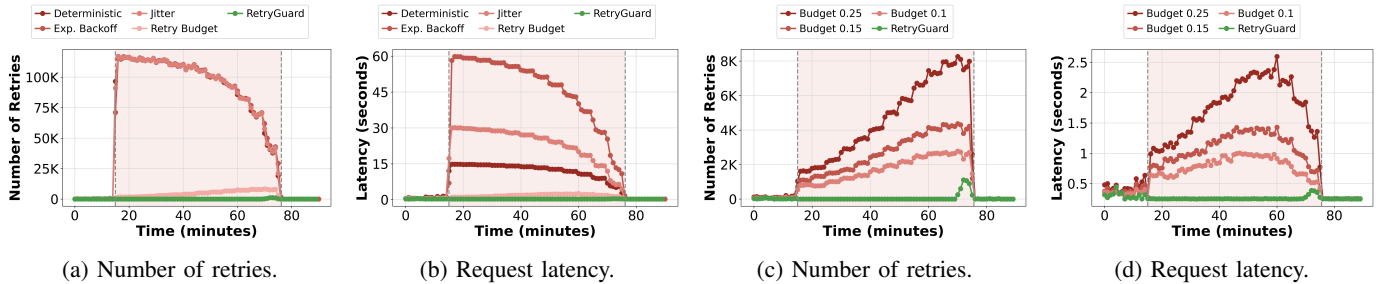


Figure 7: *RetryGuard* compared with deterministic timeouts, exponential backoff, jitter, and retry budgets. Panels (a) and (b) compare all mechanisms, while panels (c) and (d) zoom in on retry budgets under several parameter settings.

counterproductive retries, except for brief recovery probes, preserving downstream capacity for first attempts.

C. *RetryGuard* Principles, Properties and Parameters

1) *Distributed, Minimal Coordination*: *RetryGuard* requires no coordination between the source and destination services, making it ideal for distributed deployment across multiple points in a microservices graph, allowing each service to manage retries locally without the need for a centralized orchestrator. It can be implemented solely at the source without altering downstream services. Specific implementations may leverage shared observability tools or centralized metric systems (e.g., Prometheus [47] or AWS CloudWatch [31]).

2) *Seamless Operation in Coordinated (Balanced) Systems*: Importantly, *RetryGuard* is designed to be **non-disruptive** during stability conditions, ensuring seamless operation when service traffic is well-balanced and scalable. If service traffic is well-balanced, the performance metrics will not exceed the threshold (such probabilities are negligible, as analyzed in Sec. V). Thus, a sudden traffic burst will not activate intervention, and whenever a system is “ideal” or “perfectly scaled”, *RetryGuard* remains non-intrusive.

Overall, *RetryGuard* acts as a safeguard that prevents clients from compounding a prolonged period of miscoordination and overload with excessive retry attempts, while keeping the benefits of the classic retry mechanism in normal operation.

D. *RetryGuard* Implementation

1) *Surrogate Threshold Metrics*: *RetryGuard* can rely on several metrics, depending on their availability. If retry volumes per request are measurable (e.g., through retry counters), *RetryGuard* can track it to identify unproductive retries. If retry metrics are not accessible, we can instead rely on rejection signals (e.g., error logs or rejection codes). If neither is available, *RetryGuard* can use response delays (monitored via tools like AWS CloudWatch [31]) as indicators, temporarily halting retries during high latency and reintroducing them as delays normalize. *RetryGuard* is evaluated using rejection rate and response delays, to demonstrate its flexibility.¹²

¹²Sec. V introduces a mathematical framework that captures the interplay between retries, throughput and delays, offering practical threshold guidance.

2) *RetryGuard Parameter Tradeoff*: *RetryGuard* requires two parameters: an activation threshold and low/high streak durations for disabling and re-enabling retries. In the setup of Sec. VI, shorter streaks reduced unnecessary retries under light overload by over 90%, but increased the stable-load rejection rate from 1.0% to 5.3%. Under extreme DDoS load, both settings yielded similar rejection rates as performance was dominated by downstream capacity. Sec. V shows that the sharp retry-rate transition near $\rho = 1$ makes the threshold robust to configure. Both parameters remain deployment-specific and were adapted to each system’s scaling dynamics in Sec. VI.

3) *Integrating *RetryGuard**: *RetryGuard* can be deployed in both closed-source (e.g., AWS) and open-source (e.g., Istio) environments by leveraging existing observability signals, enabling adaptive retry control without modifying application code or existing request paths. In AWS, we deployed *RetryGuard* by continuously monitoring services on a dedicated EC2 instance via existing telemetry (CloudWatch) and dynamically updating service configurations, so its decisions neither block nor delay user traffic in-flight. In Istio, *RetryGuard* was implemented as a dedicated pod that periodically samples Istio’s metrics interface, imposing no per-request overhead.

V. ANALYSIS: LOAD, DELAY AND COST

In this section we use an analytical model to examine the critical transition of a service from stable-load to miscoordination and to support *RetryGuard* in two key aspects: (1) In establishing key design principles and in deriving threshold activation guidelines for the Productive-Retry Controller; and (2) In estimating the costs of retries under miscoordination, as well as retry rate, rejection probability, and expected delay.

In Sec. V-A, we formulate the mathematical framework and analyze it under miscoordination. Sec. V-B analyzes a service under stable-load conditions. In Sec. V-C, we examine the stable-load/miscoordination transition and evaluate *RetryGuard*’s robustness. Our analysis establishes that *RetryGuard* can robustly detect miscoordination (overload) while remaining non-intrusive under stable-load.

A. Retries and Rejections Under Miscoordination

The system under consideration consists of Service A and Service B operating in tandem (see Fig. 8). Service A receives

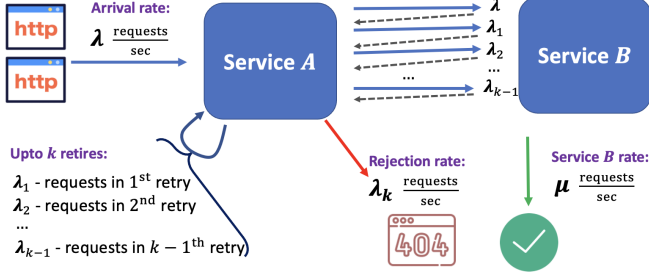


Figure 8: The Model and notation.

incoming requests at rate λ (reqs/sec). Service A holds the transactions and attempts to forward them to Service B. Let μ denote the rate at which Service B can process requests (i.e., its capacity in requests per second). If Service B cannot accept the requests due to capacity constraints, Service A will retry later. We assume that Service A imposes a limit on the number of retries per request, denoted by k . Let λ_i represent the rate of requests currently in their i -th retry attempt, where $i = 1, \dots, k$. This notation is illustrated in Fig. 8.

We analyze the steady-state behavior of the system in a miscoordination scenario, where $\rho = \lambda/\mu > 1$, using a fluid approximation of aggregate retry traffic under sustained, nearly stationary, overload. The model captures average request and retry rates rather than per-request queueing dynamics or short-timescale correlations. Fig. 9 validates it against finite-buffer $M/M/1/m$ simulations across stable and overloaded regimes. We define p as the success probability of an arbitrary request by Service B¹³. Let $\tilde{p} := 1 - p$ be the rejection probability. Note that p depends on the retries rate, as we show in Eq. (2) below.

1) *Deriving λ_i and $\tilde{p}$:* The total volume of requests that Service B receives per second is:

$$\Lambda = \lambda + \sum_{i=1}^{k-1} \lambda_i, \quad (1)$$

which includes new requests, λ , and retries, λ_i . Since the capacity of Service B is μ , and $\mu < \lambda < \Lambda$ in an overload scenario, the success probability of an arbitrary request, p , and, consequently, the rejection probability $\tilde{p}$, are given by:

$$p = \mu/\Lambda = \mu/(\lambda + \sum_{i=1}^{k-1} \lambda_i); \quad \tilde{p} = 1 - \mu/\Lambda. \quad (2)$$

Under the fluid approximation, all attempt classes share the same downstream capacity and observe the same aggregate congestion state. We therefore approximate their time-averaged rejection fractions by a common value $\tilde{p}$. Under this assumption: $\lambda_1 = \tilde{p} \cdot \lambda$ and $\lambda_i = \tilde{p} \cdot \lambda_{i-1}$ for $i = 2, \dots, k$. Expanding this relationship yields $\lambda_i = \tilde{p}^i \cdot \lambda$, for $i = 1, \dots, k$. By substituting this expression into Eq. (2) and Eq. (1), and

¹³Rejection indicates that, at that specific attempt, Service B could not process the request. During the first $k - 1$ attempts, the request will retry according to the defined retry pattern. However, on the k th attempt, the request will be considered “failed” (i.e., throttled) by Service B, as illustrated by the red arrow in Figure 8.

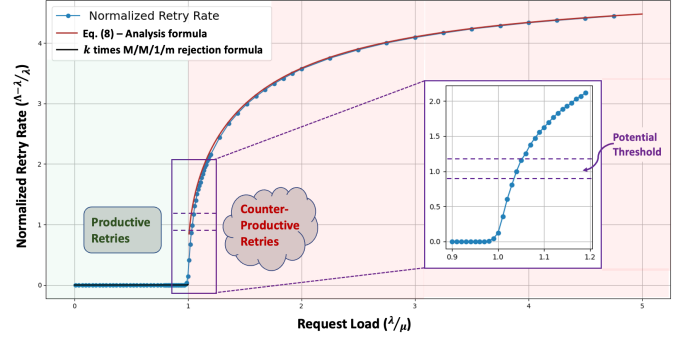


Figure 9: Normalized retry rate as a function of request load. The zoomed-in panel focuses on the critical transition range where retries rise sharply.

through algebraic simplifications (due to lack of space, we omit the intermediate steps) we derive:

$$\tilde{p} = \sqrt[k]{1 - \mu/\lambda} \quad \text{and} \quad \lambda_i \approx \lambda \cdot (1 - \mu/\lambda)^{\frac{i}{k}}. \quad (3)$$

Remark V.1. The accuracy of the analysis is verified by a comparison to simulation in Fig. 9. Examining the curves to the right of load = 1, where the red curve represents the approximation and the blue dots represent simulation results (on $M/M/1/m$), shows very good agreement.

2) *Deriving the Expected Delay $E[T]$:* Next, we derive the expected delay (or lifetime) of a request in the system, focusing on the well-known Exponential Backoff strategy for retries. While our analysis centers on Exponential Backoff, similar methods can be applied to other retry strategies, such as Jitter or Linear Increasing Delays. Retries follow a truncated geometric distribution. Let X represent the number of retries experienced by an arbitrary request. Then:

$$Pr[X = i] = \begin{cases} (1 - p)^i \cdot p & 0 \leq i \leq k - 1, \\ (1 - p)^k & i = k \end{cases} \quad (4)$$

and 0 for any $i > k$. In Exponential Backoff, the waiting time between retries increases with each attempt, following powers of 2. Let T_i denote the total delay experienced by a request with i retries: $T_i = 1 + 2 + \dots + 2^{i-1} = 2^i - 1$. Using Eq. (4), the expected delay of a request is: $E[T] = \sum_{i=0}^k Pr[X = i] \cdot T_i$. After algebraic manipulation, we obtain:

$$E[T] = \frac{2\tilde{p}p[(2\tilde{p})^k - 1]}{2\tilde{p} - 1} - \frac{\tilde{p}p[\tilde{p}^k - 1]}{\tilde{p} - 1} + \tilde{p}^k(2^k - 1). \quad (5)$$

This metric can be used to estimate the costs incurred by Service A on the applications, since they are proportional to the time spent by a request in the system (i.e., its delay).

B. Behavior under Stability Conditions

To ensure that *RetryGuard* remains non-intrusive under stable-load conditions, we analyze its behavior using the $M/M/1/m$ queueing model, which is widely known in the literature for modeling systems with finite buffer capacity. In this model, $\lambda < \mu$, ensuring a system under stability

conditions. The rejection probability of a request, $\tilde{p}$, can be expressed as:

$$\tilde{p} = (1-\rho)\rho^m / (1-\rho^{m+1}) \quad (6)$$

where m is the buffer capacity of Service B and $\rho = \lambda/\mu$.

This allows us to estimate the likelihood for a request that is submitted to Service B to be rejected and retried under stability conditions. Fig. 9 (to the left of $\rho = 1$) depicts the retry rate (black line) based on this analysis and estimated at $k\tilde{p}$, closely matching simulation results (blue dots).

C. Practical and Operational Implications

1) **RetryGuard's transitions between stability conditions and miscoordination are robust:** The analysis provided above (Eqs. (3) and (6)) establishes that the retry rate undergoes a very sharp increase as the system moves from stability conditions to miscoordination conditions. Fig. 9 demonstrates this phenomenon by depicting the normalized retry rate as a function of the request load (load due to original requests). The reader may observe that a sharp increase (steep derivative) at $\rho = 1$ marks the transition point from stable-load to over-load.

This sharp increase allows *RetryGuard* to robustly identify the transition threshold from stability conditions to miscoordination and vice versa, since simple counting of retries (or other related parameters) over a short period of time will yield a highly robust identifier. For example, one can measure the normalized retry rate and select any value between 0.95 and 1.1 (see purple-dashed lines in Fig. 9) as a transition threshold value. Due to the sharp increase of the curve, this selection is robust to both the specific threshold chosen and measurement noise. The sharp increase phenomenon occurs since at $\rho < 1$ retries occur only at rare events of blocking due to statistical variation of the arrival process. In contrast, at $\rho > 1$ at least a fraction $\rho - 1$ of the offered load is blocked, and the resulting retry traffic is amplified by a factor of k due to repeated retries.

2) **The Price Tag of The Storm:** Evaluating $E[T]$ from Eq. (5) across the system parameters λ , μ , and k reveals two key insights. First, delay and cost grow rapidly with the retry limit k : as $\tilde{p} \rightarrow 1$, exponential backoff makes the expected request lifetime exponential in k . Second, even 'low' or 'mild' overload (ρ slightly above 1) causes substantial cost increases as retries accumulate. Sec. VI confirms these analytical insights through measured latency and billing.

VI. EVALUATION

We evaluate *RetryGuard* to demonstrate its practicality, effectiveness, and advantages over existing retry patterns. The evaluation is organized as follows: In Sec. VI-A, we deploy and compare *RetryGuard* to AWS's Standard, Legacy, and Adaptive retry policies [51]. This comparison also covers the fundamental retry patterns exponential backoff, jitter, and token-bucket. In Sec. VI-B, we deploy *RetryGuard* in a multi-layer distributed application using the Kubernetes Istio Bookinfo environment, showing its improved results. Table I (Page 2) provides an overview of the main results.

A. RetryGuard and AWS Retries

We conducted several experiments using the AWS application from Sec. III. *RetryGuard* was implemented as described in Sec. IV-D. That is, the AWS Gateway-Lambda-DynamoDB application was deployed, traffic was launched at it using an EC2 instance [52], and metrics were measured with CloudWatch [31]. Each experiment was repeated three times.

We compared the performance of *RetryGuard* against the AWS Boto3¹⁴ retry patterns [51]:

- **Legacy** (default in Boto3) uses attempts with backoff, continuing to retry under most network-related exceptions.
- **Standard** applies exponential backoff with jitter, respecting maximum number of attempts and delay constraints.
- **Adaptive** uses a token-bucket approach, relying on client-side observed throughput to limit the retry rate.

We measured several key metrics to assess both cost and performance under miscoordination: (1) **Retry Storm Size** (Fig. 10a): the number of retry attempts triggered. (2) **Lambda Latency** (Fig. 10b): the average delay experienced by Lambda requests throughout the experiment. (3) **Number of Successful Requests** (Fig. 10c): the number of requests ultimately succeeded (whether on the first attempt or after retries), reflecting overall throughput. (4) **Number of Scaling Operations** (Fig. 10d, left): the number of times DynamoDB scaled up. (5) **Provisioned Capacity** (Fig. 10d, right): the final capacity provisioned by the DynamoDB, which directly impacts cost.

The measurements begin when traffic increases and miscoordination occurs, and continue until the system scales and reaches a stable state. Fig. 10a shows that *RetryGuard* substantially reduces the size of the retry storm, eliminating many unnecessary retry attempts. Specifically, *RetryGuard* lowered the average number of retry attempts per request from 2.09 (with the default Legacy pattern) to only 0.05, representing a 98% decrease. Subsequently, as shown in Fig. 10b, *RetryGuard* achieves significantly lower end-to-end latency than all other AWS retry patterns, resulting in substantial cost savings ranging from 40% to 90% just for the Lambda billing.

Fig. 10d illustrates that *RetryGuard* minimized scaling costs while stabilizing the system. It reduces the number of scaling operations, and eliminates over-scaling, as without it DynamoDB's provisioned capacity exceeds actual needs.

Despite these improvements, Fig. 10c shows that *RetryGuard* achieves a similar number of successful writes to the DB compared to all other retry patterns. Naturally, a portion of requests fail due to DynamoDB overload, but additional retries beyond saturation do not improve effective throughput, resulting in a similar overall rejection rate for all patterns. Hence, while *RetryGuard* delivers the best performance, it also yields the lowest costs.

B. Multi-layer Distributed Deployment

We evaluate *RetryGuard* in a multi-layer environment using *Bookinfo* (Sec. III). We deployed *RetryGuard* in a distributed manner, running independently at each microservice. The

¹⁴The official AWS Software Development Kit for Python.

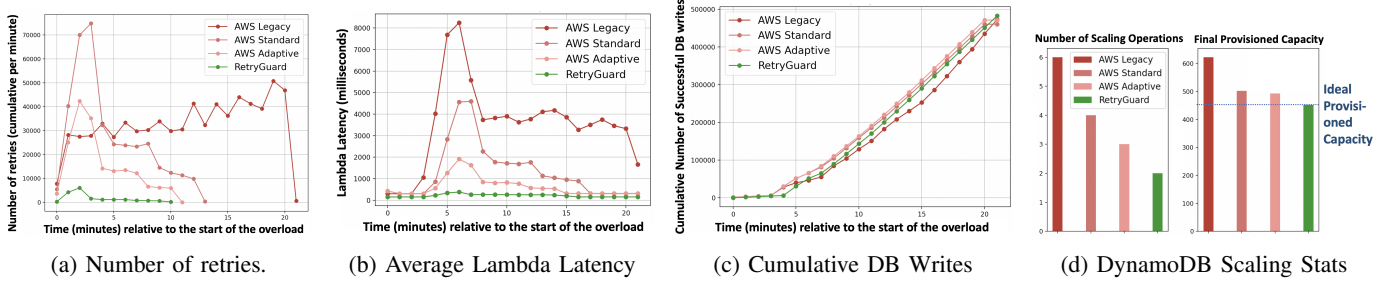


Figure 10: *RetryGuard* achieves lower retry rate and latency while maintaining the rejection rate (DB writes).

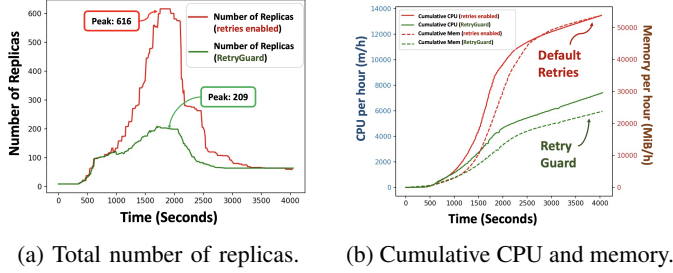


Figure 11: Results from the Istio Bookinfo experiments.

deployment leverages the service mesh framework, which exposes the rejection rate of each service. If the rejection rate exceeds the threshold (20% in the experiment) for an interval of 30 seconds, retries are disabled for every request at that service; once it drops below the threshold, retries are *gradually* re-enabled again. We use the same experimental setup, metrics, and load-generation method described in Sec. III.

Enabling *RetryGuard* effectively prevents the retry storm that occurs when retries are enabled. With *RetryGuard*, the number of requests received by the *Reviews* service stabilizes, and the service itself scales up more moderately. Although only *Reviews* was configured with a slower HPA, *RetryGuard*'s benefits of containing the retry storm propagated to the entire system, including the *Ratings* and *Product* services. *RetryGuard* further lowers total resource consumption, as reflected in the lower number of replicas, CPU usage, and memory usage. With *RetryGuard*, the system peaks at 209 replicas, compared with 616 replicas without it (see Fig. 11a), a nearly $3\times$ reduction in peak replica count. Over the entire experiment, *RetryGuard* also reduces cumulative memory usage by 55% (see Fig. 11b).

Performance actually improves: under the default configuration, the rejection rate was 5%, and the average latency was 5.17 seconds. With *RetryGuard*, the rejection rate was reduced to 4% and latency dropped to 4.02 seconds. Hence, we reduced both time and cost, without compromising reliability.

We also observed that *RetryGuard* only disabled retries for the *Reviews* service, which frequently encountered errors. Meanwhile, the *Product*, *Details*, and *Ratings* services maintained retries almost continuously, preserving the benefits of retry mechanisms in these services.

VII. DISCUSSION

Malicious Attacks: Short DDoS Burst, Long-Term DoW.

Applications that employ retry patterns are at significant risk of substantial economic damage from DDoS attacks. Recently, attacks involving short, repeating waves have become increasingly common, confounding conventional DDoS mitigation and detection solutions [53]–[55]. In many cases, the goal is to strain the budget of the victim. When retries are enabled, any miscoordination in scaling is likely to trigger a self-inflicted DoW “snowball” scenario that persists for minutes after each short-burst attack, even if the attack itself lasts only a few seconds. As the attacker halts the attack burst, malicious requests remain in the system, forming a queue of pending requests. Consequently, new incoming requests are impacted and conflict with the retried malicious requests, creating a rolling effect of queued requests. This cascading issue persists until the system scales up and clears the queue. See Fig. 12.

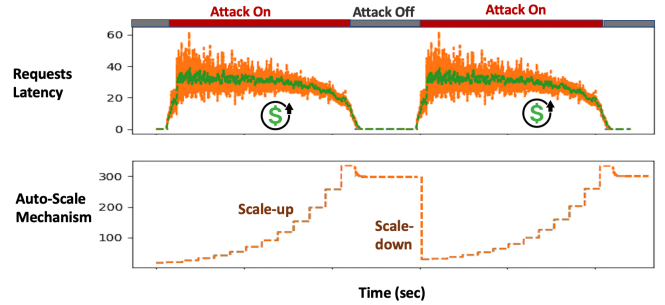


Figure 12: Burst-DDoS: Retry storms amplify the attack's impact and may be repeated to exhaust the victim's budget.

Using *RetryGuard*, the system detects overload within one interval and prevents retries from amplifying the attack beyond the attacker's own traffic.

Economic Interplay. It is important to note that the overall operational cost of microservices in the cloud is a rather intricate issue. This results from the interplay between the economic players. The cloud clients (namely application owners) find the serverless services approach attractive as they seek to concentrate on the application level and avoid getting into the details of the services. However, as we demonstrate, this approach may expose application owners to significant added

costs, stemming from the complex interactions among their microservices. Cloud providers, on the other hand, may have little incentive to address this problem, as the excess usage it generates ultimately adds to users' bills, and thus to their own revenues.

VIII. CONCLUDING REMARKS

We introduced *RetryGuard*, a distributed framework that suppresses counterproductive retries during prolonged miscoordination. Our analysis and evaluation show that *RetryGuard* substantially reduces resource consumption and cost. In our AWS Serverless deployment, it achieves a 98% reduction in storm size and more than a 90% reduction in latency; in our Kubernetes Istio deployment, it reduces peak replicas by 3× and cumulative memory usage by 55%. Future work can explore broader forms of regime-aware retry control.

REFERENCES

- [1] Y.-H. Lee and K. G. Shin, "Optimal design and use of retry in fault-tolerant computer systems," *Journal of the ACM (JACM)*, vol. 35, no. 1, pp. 45–69, 1988.
- [2] "Retry storm antipattern," <https://learn.microsoft.com/en-us/azure/architecture/antipatterns/retry-storm/>, 2022.
- [3] A. H. Dave Rensin, "How to avoid a self-inflicted ddos attack—cre life lessons," <https://cloud.google.com/blog/products/gcp/how-to-avoid-a-self-inflicted-ddos-attack-cre-life-lessons>, 2016.
- [4] Y. Gan and C. Delimitrou, "The architectural implications of cloud microservices," *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.
- [5] N. Kratzke, "A brief history of cloud application architectures," *Applied Sciences*, vol. 8, no. 8, p. 1368, 2018.
- [6] J. Rahman and P. Lama, "Predicting the end-to-end tail latency of containerized microservices in the cloud," in *2019 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2019, pp. 200–210.
- [7] J. Xing, H. M. Demoulin, K. Kallas, and B. C. Lee, "Charon: A framework for microservice overload control," in *Proc. of ACM HotNets'21*, 2021.
- [8] K. Grewal, P. B. Godfrey, and J. Hsu, "Expressive policies for microservice networks," in *Proc. of ACM HotNets'23*, 2023.
- [9] A. Mampage, S. Karunasekera, and R. Buyya, "A holistic view on resource management in serverless computing environments: Taxonomy and future directions," *ACM Computing Surveys (CSUR)*, vol. 54, no. 11s, pp. 1–36, 2022.
- [10] W. Zheng, R. Bianchini, and T. D. Nguyen, "Automatic configuration of internet services," in *Proc. of EuroSys'07*, 2007.
- [11] Linkerd, "Retries and timeouts: How retries can go wrong." [Online]. Available: <https://linkerd.io/2.15/features/retries-and-timeouts>
- [12] B.-J. Kwak, N.-O. Song, and L. E. Miller, "Performance analysis of exponential backoff," *IEEE/ACM transactions on networking*, vol. 13, no. 2, pp. 343–355, 2005.
- [13] M. Brooker, "Timeouts, retries, and backoff with jitter," *The Amazon Builders' Library*, 2019.
- [14] Linkerd, "Configuring retries." [Online]. Available: <https://linkerd.io/2.15/tasks/configuring-retries/>
- [15] M. Butto, E. Cavallero, and A. Tonietti, "Effectiveness of the 'leaky bucket' policing mechanism in atm networks," *IEEE Journal on selected areas in communications*, vol. 9, no. 3, pp. 335–342, 1991.
- [16] Azure, "Retry pattern." [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/retry>
- [17] R. Caron, "Using the retry pattern to make your cloud application more resilient." [Online]. Available: <https://azure.microsoft.com/en-us/blog/using-the-retry-pattern-to-make-your-cloud-application-more-resilient/>
- [18] "Exponential backoff," <https://cloud.google.com/memorystore/docs/redis/exponential-backoff>, 2025.
- [19] D. Yu, Y. Jin, Y. Zhang, and X. Zheng, "A survey on security issues in services communication of microservices-enabled fog applications," *Concurrency and Computation: Practice and Experience*, vol. 31, 2019.
- [20] N. Lazarev, S. Xiang, N. Adit, Z. Zhang, and C. Delimitrou, "Dagger: efficient and fast rpcs in cloud microservices with near-memory reconfigurable nics," in *Proc. of ACM ASPLOS'21*, 2021.
- [21] Bookinfo application. [Online]. Available: <https://istio.io/latest/docs/examples/bookinfo/>
- [22] M. R. Saleh Sedghpour, C. Klein, and J. Tordsson, "An empirical study of service mesh traffic management policies for microservices," in *Proc. of ACM/SPEC ICPE'22*, 2022.
- [23] S. Ashok, P. B. Godfrey, and R. Mittal, "Leveraging service meshes as a new network layer," in *Proc. ACM HotNets'21*, 2021.
- [24] Cloud Native Computing Foundation (CNCF), "Service meshes are on the rise—but greater understanding and experience are required," 2022.
- [25] L. M. Vaquero, L. Roderio-Merino, and R. Buyya, "Dynamically scaling applications in the cloud," *ACM SIGCOMM Computer Communication Review*, vol. 41, no. 1, pp. 45–52, 2011.
- [26] Y. Liu, N. Rameshan, E. Monte, V. Vlassov, and L. Navarro, "Prorenata: Proactive and reactive tuning to scale a distributed storage system," in *Proc. of IEEE/ACM CCGrid'15*, 2015.
- [27] T. Chen, R. Bahsoon, and X. Yao, "A survey and taxonomy of self-aware and self-adaptive cloud autoscaling systems," *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–40, 2018.
- [28] O. Michaelis, S. Schmid, and H. Mostafaei, "L3: Latency-aware load balancing in multi-cluster service mesh," in *Proceedings of the 25th International Middleware Conference*, 2024, pp. 49–61.
- [29] A. Bremner-Barr, M. Czeizler, H. Levy, and J. Tavori, "Exploiting miscoordination of microservices in tandem for effective ddos attacks," in *IEEE INFOCOM 2024*, 2024, pp. 1–10.
- [30] M. Walterbusch, B. Martens, and F. Teuteberg, "Evaluating cloud computing services from a total cost of ownership perspective," *Management Research Review*, vol. 36, no. 6, pp. 613–638, 2013.
- [31] "Amazon CloudWatch," <https://aws.amazon.com/cloudwatch>, 2025.
- [32] R. Iyer, V. Tewari, and K. Kant, "Overload control mechanisms for web servers," in *Proc. of the International Conference on the Performance and QoS of Next Generation Networking, P&QNet'00*, 2000.
- [33] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM computer communication review*, vol. 18, no. 4, pp. 314–329, 1988.
- [34] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu, R. Gu, B. C. Ooi, and J. Yang, "Overload control for scaling wechat microservices," in *Proc. of ACM SoCC'18*, 2018.
- [35] I. Cho, A. Saeed, J. Fried, S. J. Park, M. Alizadeh, and A. Belay, "Overload control for {μs-scale}{RPCs} with breakwater," in *Proc. USENIX OSDI'20*, 2020.
- [36] J. Xing, A. Giannoukos, P. Loh, S. Wang, J. Qiu, H. M. Demoulin, K. Kallas, and B. C. Lee, "Rajomon: Decentralized and coordinated overload control for {Latency-Sensitive} microservices," in *Proc. USENIX NSDI'25*, 2025.
- [37] J. Park, J. Park, Y. Jung, H. Lim, H. Yeo, and D. Han, "Topfull: An adaptive top-down overload control for slo-oriented microservices," in *Proc. of ACM SIGCOMM'24*, 2024.
- [38] I. Cho, A. Saeed, S. J. Park, M. Alizadeh, and A. Belay, "Protego: Overload control for applications with unpredictable lock contention," in *Proc. of USENIX NSDI'23*, 2023.
- [39] M. Welsh and D. Culler, "Overload management as a fundamental service design primitive," in *Proceedings of the 10th workshop on ACM SIGOPS European workshop*, 2002, pp. 63–69.
- [40] L. Huang, M. Magnusson, A. B. Muralikrishna, S. Estyak, R. Isaacs, A. Aghayev, T. Zhu, and A. Charapko, "Metastable failures in the wild," in *Proc. of USENIX OSDI'22*, 2022.
- [41] N. C. Mendonca, C. M. Aderaldo, J. Cámara, and D. Garlan, "Model-based analysis of microservice resiliency patterns," in *Proc. IEEE ICSE'20*. IEEE, 2020, pp. 114–124.
- [42] J. Tavori, A. Bremner-Barr, and H. Levy, "The cost of retrying: Exploiting retry-mechanisms in cloud applications by ddos attacks," in *Proc. of Companion of CoNEXT'23*, 2023.
- [43] "AWS lambda documentation, 2023," <https://docs.aws.amazon.com/lambda>, 2026, accessed: 2026-06-02.
- [44] "AWS dynamodb documentation," <https://docs.aws.amazon.com/dynamodb>, 2026, accessed: 2026-05-01.
- [45] Istio, "Traffic management: Retries," <https://istio.io/latest/docs/concepts/traffic-management/#retries>, istio Documentation.
- [46] (2025) Horizontal pod autoscaling. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [47] B. Rabenstein and J. Volz, "Prometheus: A next-generation monitoring system," in *SREcon15 Europe*. USENIX Association, 2015.

- [48] Y. Verginadis, "A review of monitoring probes for cloud computing continuum," in *International Conference on Advanced Information Networking and Applications*. Springer, 2023, pp. 631–643.
- [49] Google Cloud, "Google cloud monitoring documentation," 2026. [Online]. Available: <https://cloud.google.com/monitoring/docs>
- [50] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time," *Queue*, vol. 14, pp. 20–53, 2016.
- [51] Boto3 Documentation, "Boto3 documentation – retries," <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/retries.html>.
- [52] U. guide, "What is amazon ec2 auto scaling?" [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html>
- [53] A. G. Alcoz, M. Strohmeier, V. Lenders, and L. Vanbever, "Aggregate-based congestion control for pulse-wave ddos defense," in *Proc. of ACM SIGCOMM 2022*, 2022, pp. 693–706.
- [54] M. Sides, A. Bremler-Barr, and E. Rosensweig, "Yo-yo attack: Vulnerability in auto-scaling mechanism," *SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, p. 103–104, aug 2015.
- [55] Q. Li, H. Huang, R. Li, J. Lv, Z. Yuan, L. Ma, Y. Han, and Y. Jiang, "A comprehensive survey on ddos defense systems: New trends and challenges," *Computer Networks*, p. 109895, 2023.